

Basics101/112

AI Detail Usage0/182

Links0/21

Build Provenance0/20

Team5/7

- Basics
- Team
- AI Detail Usage
- Links2
- Build Provenance2
- Audit Trail

Project Info

Project Name *

EcoSortha AI

Elevator Pitch *

An AI-native Resource Intelligence Platform that converts organic bio-assets into IoT-verified "Liquid Nutrients" and "Carbon Enh

Public Summary *

EcoSortha AI is a circular economy marketplace and SME intelligence platform built for Bangladesh's organic agriculture sector. Less than 3% of the country's BDT 800 crore organic fertilizer market is verifiably certified — leaving nursery owners and farmers unable to distinguish genuine organic products from synthetic imitations.

Domain

Online Commerce (E-Commerce)

Challenge

SME Dashboard

Problem Statement *

What problem does it solve?

Bangladesh's organic fertilizer market is worth BDT 800 crore annually, yet less than 3% of products carry any verifiable

1. THE TRUST GAP: Green SMEs have no affordable digital tool to prove bio-product quality. The only verification option is a third-party lab audit costing BDT 15,000–40,000 per batch — unaffordable for most small producers.

Solution Description *

How does your solution work?

EcoSorta AI is a 7-layer, AI-native Resource Intelligence Platform operating as a PaaS (Platform-as-a-Service) for Green SMEs. It transforms organic bio-assets through a three-stage intelligence pipeline:
STAGE 1 — MULTIMODAL FEEDSTOCK GRADING (Input Layer)
A MobileNetV2 image classifier (TensorFlow) analyzes incoming feedstock photos to determine crop quality (rot, pest, mold, etc.). (Data for fertilizer recommendation is generated here).



Data Lifecycle & Engineering

Show your full AI-native data stack — acquisition, parsing, storage, visualization, ML/insights, outbound APIs, and governance.

1. Data Sources

Where does your data come from? Select all that apply.

- ☒ Internal (own DB / app data)
- ☒ External APIs (paid/free)
- ☒ Public Web (scraping)
- ☒ Open Datasets (Kaggle, HF, gov)
- ☒ User Uploads / Bulk Import
- ☒ IoT / Sensor / Streaming
- ☐ Third-party / Partner Data
- ☐ Synthetic / AI-generated Data

Internal: Supabase PostgreSQL — batch records, IoT sensor readings, order history, buyer profiles, trust certificates, demand forecasts.
IoT Sensors: ESP32 microcontroller with pH electrode sensor, EC conductivity probe, DS18B20 digital thermometer — real-time readings via HTTP POST to backend API.

 2. Acquisition Methods

How do you collect the data?

☒ Web Scrapers (Playwright, Puppeteer, Scrapy)	☒ AI Extraction (LLM parsing of unstructured)
☐ MCP Servers for data access	☐ Bulk Upload (CSV/XLSX/PDF intake)
☒ Automated Flows (n8n, Airflow, cron, webhooks)	☒ API Pull / SDK integrations
☐ RSS / Atom Feeds	☐ Email Inbox / Inbound Parsing
☐ OCR (Tesseract, Donut, Textract)	☐ Speech-to-Text (Whisper, Deepgram)

Scrapers / crawlers used

Firecrawl: scrapes Daraz.com.bd and Chaldal.com product pages for organic fertilizer pricing data. Configured for weekly automated runs via Railway cron (every Monday).

knowledge_embeddings PGvector table

IoT automated flow: ESP32 sensor reads every 30 minutes → HTTP POST to /api/iot/reading → backend validates reading → recalculates Trust Score →

MCP servers for data access

e.g. Postgres MCP, Filesystem MCP, custom MCP exposing internal warehouse...

 3. Parsing, Formats & Cleaning

Raw formats handled and how you normalize them.

JSON

CSV

XLSX

PDF

Parquet

Avro

XML

YAML

JSONL

Markdown

HTML

Images

Audio

Video

Protobuf

Parsers used

PyPDF2: extracts text from BARI and WHO standard PDF documents for RAG chunking.

BeautifulSoup4: parses HTML from scraped Daraz/Chaldal product pages

Formatters / converters

jsPDF: generates batch Trust Certificates and ESG Impact Reports as downloadable

PDF files. QRCode.js generates QR code images embedded in the PDFs

Data cleaning & enrichment

IoT reading validation: backend validates pH (0–14 range), EC (0–2000 µmhos/cm)

Schema validation

Zod (TypeScript): validates all API request bodies in Express routes before

4. Storage Targets

☒ Relational (Postgres, MySQL)

☐ NoSQL (Mongo, Dynamo, Firestore)

☒ Vector DB (pgvector, Pinecone, Weaviate)

☒ Object Storage (S3, R2, GCS)

☐ Data Warehouse (BigQuery, Snowflake, DuckDB)

☐ Lakehouse (Delta, Iceberg, Hudi)

☐ Graph DB (Neo4j, ArangoDB)

☐ Cache / KV (Redis, Memcached)

10 core tables in Supabase PostgreSQL:

users (UUID PK, role enum, language preference, credits)

feedstock_logs (image_url, ai_class, confidence, weight_kg, status pipeline)

batches (processor_id, input_kg, output_liters, trust_score, status, cert URL)

5. Visualization (open source preferred)

☒ Recharts

☐ Chart.js

☐ D3.js

☐ Plotly

☐ Apache ECharts

☐ Vega/Vega-Lite

☐ Observable

☐ Superset

☐ Metabase

☐ Grafana

☐ Kibana

☐ Streamlit

☐ Dash

Add another viz tool...

+ Add

Visualization details

All charts built with Recharts in Next.js components. Dark theme throughout

(background #1a1a1a, accent green #4CAF7D, ash #C6A992). Responsive — tested

Dashboards & reports

Processor Dashboard: Live Trust Score + demand forecast + ESG impact metrics.

Buyer Marketplace: Product grid with Trust Score badges + AI recommendation card

6. Insights — AI, ML & Non-AI

☐ Classical ML (scikit-learn, XGBoost, LightGBM)

☒ Deep Learning (PyTorch, TensorFlow, JAX)

☒ LLM Inference / RAG over data

☒ Forecasting (Prophet, statsmodels, NeuralProphet)

☐ Anomaly Detection

☐ Clustering / Segmentation

☒ Rule Engine / Heuristics (non-AI)

☐ Statistical Analysis

AI / ML details

Model 1 — MobileNetV2 (TensorFlow/Keras):

Task: Binary feedstock classification (Wet organic / Dry recyclable)

Input: RGB image 224x224 pixels, normalized to [0,1]

Non-AI analytics

Dynamic pricing formula: $\text{base_price} \times \text{demand_factor} \times \text{trust_factor} \times \text{supply_factor}$
demand_factor: 1.15 if forecast > stock×1.5 (high demand signal)

How are insights delivered to users?

Processor Dashboard (Next.js): Real-time Trust Score gauge updates as IoT readings are entered (no page refresh — Supabase Realtime subscription). 30-day demand forecast displayed as Recharts AreaChart with peak demand highlighted and AI nudge ("Start next batch in X days"). ESG Impact Card shows live plastic offset and carbon

7. Pipelines & Orchestration

Orchestration

Railway cron jobs (pg_cron equivalent) handle weekly pipeline:
Step 1: Firecrawl scrape → price normalization → Supabase insert

Scheduling / Triggers

Weekly cron: Monday 6:00 AM (price scrape + forecast refresh)
IoT event trigger: ESP32 POST → immediate Trust Score recalculation

Streaming / Real-time

Supabase Realtime: processor dashboard subscribes to batches table changes.
Trust Score updates pushed to

8. Outbound — APIs & Distribution

Outbound APIs

Public REST API (Express): all endpoints documented with JSDoc comments.
Authentication: JWT Bearer token

Webhooks & exports

bKash payment webhook: POST /api/payments/bkash/callback → verifies payment
taken → updates order

Embeddings / model serving

MobileNetV2: served as Flask microservice (Railway) at /classify endpoint.
Prophet: pre-computed forecast

🔗 8. Outbound — APIs & Distribution

Outbound APIs

Public REST API (Express): all endpoints documented with JSDoc comments.
Authentication: JWT Bearer token

Webhooks & exports

bKash payment webhook: POST /api/payments/bkash/callback → verifies payment
taken → updates order

Embeddings / model serving

MobileNetV2: served as Flask microservice (Railway) at /classify endpoint.
Drophet: pre-computed forecasts

🔗 9. Open Source Stack

List the open-source tools you used across the data lifecycle and what role each played.

Frontend (VUE), LangChain, LangChain-OpenAIWrapper, OpenAI, OpenAI GPT-4o, Supabase (Apache 2.0): PostgreSQL + PGVector + Storage + Realtime.
Pillow (HPND): Image pre-processing for MobileNetV2 input pipeline.
QRCode.js (MIT): QR code generation for batch certificates.
Zod (MIT): TypeScript schema validation for all API request bodies

🛡️ 10. Quality, Governance & Observability

Data quality

IoT reading validation: Zod schemas enforce physical ranges (pH 0–14, EC 0–20, temp 0–100) at API layer before any database write.
Trust Score formula is deterministic — same inputs always

Privacy & compliance

Batch certificates show anonymized Processor ID (e.g. "Certified Processor #07")
— never processor name or personal details — on the public QR verify endpoint

🔄 Lineage & observability

Every batch record has a full audit trail: created_at timestamp, processor_id, all IoT readings (separate table), Trust Score history, and certification event

💰 Cost & performance

Claude API calls: rate-limited to 10 requests/min per user.
RAG retrieval (top-5 chunks) reduces token count vs full-document prompting.
Average LLM call: 400

📌 Anything else about your data stack?

NOVEL ARCHITECTURE — IoT-to-Certificate pipeline:
The most distinctive aspect of EcoSortha AI's data stack is the deterministic Trust Score engine sitting between raw IoT sensor readings and a verifiable PDF certificate. Unlike ML-based quality scoring (which has bias risk and



AI Detail Usage

Tell us how AI shaped your build — prompts, models, retrieval, tools, and protocols. Each section earns points that aggregate into your AI Depth Score.



AI DEPTH SCORE

Auto-calculated as you fill each section

0/110

0%

Prompt Usage

+0 /10

How did you design prompts? Patterns used (few-shot, chain-of-thought, role prompting), iteration approach, prompt versioning.

e.g. We used structured XML prompts with role definitions, chain-of-thought reasoning for complex queries, and maintained a prompt registry...

Token Optimization

+0 /10

Strategies to reduce cost & latency: caching, summarization, context trimming, smaller models, batching, structured outputs.

e.g. We used prompt caching for system instructions, summarized chat history beyond N turns, routed simple queries to a smaller model...

LLMs / Models Used

+0 /15

Select all models used. Add others not in the list. Each model adds +3 (max 15).

☐ Claude

☐ ChatGPT

☐ Gemini

☐ Kimi

☐ DeepSeek

☐ Llama

☐ Mistral

☐ Grok

☐ Qwen

Add another model (e.g. Phi-3, Cohere Command R)...

+ Add

Add another model (e.g. Phi-3, Cohere Command R)...

+ Add

How & why did you use these LLMs? +0 /5

e.g. Claude Sonnet for code generation & reasoning, Gemini Flash for cheap summarization, DeepSeek for math-heavy queries...

Retrieval & RAG +0 /12

Select all retrieval techniques used. Each adds +3 (max 12). Leave blank if none.

☐ Naive RAG (chunk + embed + retrieve)

☐ Vector Database (Pinecone, Weaviate, pgvector, etc.)

☐ Contextual RAG (Anthropic-style, +context per chunk)

☐ Variable / Semantic Chunking

☐ Late Chunking (embed long → split)

☐ Graph RAG

☐ Knowledge Graph / Other Graph Methods

☐ Hybrid Search (Keyword + Vector)

☐ Rerankers (Cohere, BGE, etc.)

☐ Agentic RAG / Multi-step Retrieval

☐ Self-RAG (self-reflective retrieval)

☐ Corrective RAG (CRAG)

☐ Query Rewriting / HyDE

Add another retrieval technique...

+ Add

RAG architecture details (data sources, chunking, embeddings, index) +0 /5

e.g. pgvector with text-embedding-3-small, semantic chunking at 512 tokens, hybrid BM25 + vector with Cohere reranker...

MCP (Model Context Protocol) Usage

+0 /20

Inventory every MCP server you **built** and every MCP server you **used**. Capture endpoint counts, transports, and reuse across projects so judges can verify protocol depth.

☐ We built and/or used MCP servers / clients in this build

Open Source Tools & Libraries

+0 /8

List open-source projects you leveraged (LangChain, LlamaIndex, Ollama, vLLM, etc.) and how each contributed.

e.g. LangGraph for agent orchestration, Ollama for local Llama 3 inference, Unstructured.io for document parsing...

Agent Frameworks & Orchestration

+0 /7

Multi-agent setups, tool calling, planners, schedulers (LangGraph, CrewAI, AutoGen, custom orchestration).

e.g. CrewAI with 3 specialized agents (researcher, writer, critic) coordinated via a supervisor pattern...

Fine-tuning / Adaptation

+0 /5

LoRA, QLoRA, full fine-tunes, distillation, instruction tuning, embedding fine-tuning. Skip if not applicable.

e.g. LoRA fine-tuned Llama 3 8B on 2k domain Q&A pairs for medical terminology...

Evaluation & Quality Measurement

+0 /7

How did you measure AI output quality? (LLM-as-judge, human eval, RAGAS, custom benchmarks, regression sets.)

e.g. RAGAS for retrieval faithfulness, GPT-4 as judge for response quality, 50-question regression test on each prompt change...

Guardrails, Safety & Privacy

+0 /6

PII redaction, content moderation, jailbreak protection, output validation, hallucination mitigation.

e.g. Input PII scrubbing with Presidio, output schema validation with Zod, refusal patterns for out-of-scope queries...

Frontend AI / Visual App Builders

+1 pt each (max 5)

AI-driven UI/app builders used to ship the front end (Lovable, v0, Bolt, Cursor Composer, Claude Artifacts, Gemini Canvas, etc.).

☐ Lovable

☐ v0 (Vercel)

☐ Bolt.new

☐ Cursor Composer / Agent

☐ Claude Artifacts

☐ Gemini Canvas

☐ ChatGPT Canvas

☐ Windsurf

☐ Replit Agent

☐ Framer AI

Add another visual AI builder...

+ Add

How did you use these tools? What % of UI was AI-built? Any custom workflows?

Workflow Automation

+1 pt each (max 4) · n8n bonus +2

Automation platforms wired into your AI stack (n8n, Zapier, Make, Airflow, LangGraph, Temporal, etc.).

☐ n8n (self-hosted workflow automation)

☐ Zapier

☐ Make (Integromat)

☐ Apache Airflow

☐ Temporal

☐ Dagster

Workflow Automation

+1 pt each (max 4) · n8n bonus +2

Automation platforms wired into your AI stack (n8n, Zapier, Make, Airflow, LangGraph, Temporal, etc.).

☐ n8n (self-hosted workflow automation)

☐ Zapier

☐ Make (Integromat)

☐ Apache Airflow

☐ Temporal

☐ Dagster

☐ Prefect

☐ LangGraph

☐ Windmill

☐ Activepieces

Add another automation tool...

+ Add

Which workflows are automated? Triggers, integrations, agents involved...

Local / On-device LLMs

Runtimes +1 each · Ollama bonus +2 · Models +2 each (max 8)

Run open-weight LLMs locally — for cost, privacy, latency, or offline. Pick the runtime(s) and the specific model(s).

Runtimes

☐ Ollama

☐ LM Studio

☐ vLLM

☐ llama.cpp

☐ Apple MLX

☐ GPT4All

☐ Text Generation Inference (TGI)

☐ TensorRT-LLM

Add another runtime...

+ Add

Local models you actually ran

- | | |
|---|---|
| ☐ Kimi (Moonshot) | ☐ DeepSeek (R1, V3, Coder) |
| ☐ GLM (ChatGLM, GLM-4) | ☐ Llama 3 / 3.1 / 3.2 |
| ☐ Qwen 2 / 2.5 | ☐ Mistral / Mixtral |
| ☐ Phi-3 / Phi-4 | ☐ Gemma 2 |
| ☐ Yi | |

Add another local model (e.g. DeepSeek-R1-Distill-7B)...

+ Add

Hardware (GPU/CPU/Apple Silicon), quantization (Q4_K_M, GGUF), why local vs cloud...



Build a Live `/docs` Module (Recommended)

Optional but highly encouraged. A `/docs` page acts as your live pitch deck + technical whitepaper + system dashboard, giving **investors and technical evaluators** a fast, credible way to assess your product, team, and architecture in minutes.

- ☐ Yes — we will run the `/docs` module prompt and ship a live documentation page
- Helps judges and investors evaluate your application faster, more fairly, and with greater trust in your data.

[Show instructions & copy prompt](#)

Anything else about your AI usage?

Anything novel or notable we didn't ask about...



Pre-Judgment Score

[Auto](#)

Submission richness — fill more sections to raise your score before judges review.

106

/342

31% complete

Basics

101/112

AI Detail Usage

0/182

Links

0/21

Build Provenance

0/20

Team

5/7

BASICS

- ☒ Project name filled 1/1
- ☒ Elevator pitch (≥20 chars) 244 chars 2/2
- ☒ Public summary (tiered length) 1320 chars 3/3
- ☒ Domain selected 1/1
- ☒ Challenge selected 1/1
- ☒ Problem statement (tiered length) 1013 chars 10/10
- ☒ Solution description (tiered length) 2019 chars 10/10
- ☒ Data lifecycle filled 5/5
- ☐ Graph DB used (Neo4j, ArangoDB, etc.) Not selected 0/5
- ☒ Vector DB used (pgvector, Pinecone, etc.) Selected 3/3
- ☐ Lakehouse / Warehouse (Delta, Iceberg, BigQuery) Not selected 0/3
- ☐ Multiple advanced scrapers / extractors (1 pt each) 3 advanced methods (cap 4) 3/4
- ☒ Advanced parsing (parsers / formatters / cleaning detailed) 4/4 parsing dimensions documented 4/4
- ☒ Data sources selected 6 selected (need 1) 2/2
- ☒ Data source details 1076 chars 2/2
- ☒ Acquisition methods selected 4 selected (need 1) 2/2
- ☒ Acquisition details 574 chars 2/2
- ☒ Scrapers / crawlers used 791 chars 2/2
- ☐ MCP servers for data access Empty 0/2
- ☒ Parsers used 517 chars 2/2
- ☒ Formatters / converters 414 chars 1/1

 Build Provenance Transparency & Auditability

 Data & AI Provenance * Judge + Admin

Data sources, AI models used, responsible AI practices

Data Sources

List data sources used (APIs, datasets, scraped data...)

AI Models

AI models used (GPT-4, Gemini, custom models...)

Responsible AI

Responsible AI practices (bias mitigation, transparency...)

>_ Tooling & IDE * Team Only

IDE, frameworks, deployment method

IDE / Editor

Select IDE...



Deployment Method

Vercel, AWS, Docker...

>_ Tooling & IDE *

Team Only

IDE, frameworks, deployment method

IDE / Editor

Select IDE...

▼

Deployment Method

Vercel, AWS, Docker...

Frameworks & Libraries

React, TensorFlow, LangChain...

Context / Memory Files

.cursorrules, CLAUDE.md, etc.

 MCP Usage

Team Only

Model Context Protocol usage details

Mcp Servers

MCP servers used (name, purpose)

Tools Exposed

Tools exposed via MCP

Permissions

Permissions granted to MCP tools

MCP Usage

Team Only

Model Context Protocol usage details

Mcp Servers

MCP servers used (name, purpose)

Tools Exposed

Tools exposed via MCP

Permissions

Permissions granted to MCP tools

Prompt Library

0 prompts

Team Only

Prompts used during development

No prompts recorded yet.

+ Add Prompt

 **You cannot submit yet. Please fill in all required fields in the submission form.**

Missing: Data & AI Provenance, Tooling & IDE, YouTube Video, Demo / Live Links

 Judging Criteria 6 criteria

① Each criterion has its own score range (shown beside it). Final score is a weighted total normalized to **0–100**. Minimum **5** judges per submission.

1 Innovation

0–20

20%

Originality, creativity, and non-obvious AI application. Judges evaluate whether the idea is meaningfully differentiated, applies AI in a transformative way, and demonstrates bold or systems-level thinking rather than incremental improvements.

Score bands:

- 0–5: Incremental improvement or common idea.
- 6–10: Moderately differentiated concept with limited novelty.
- 11–15: Strong originality and creative AI integration.
- 16–20: Breakthrough or globally competitive innovation.

2 Technical Execution

0–20

20%

Architecture quality, AI integration depth, system robustness. Judges assess the clarity and robustness of the technical architecture, including input → processing → output workflow, model selection logic, integration of AI technologies (LLMs, ML, RAG, multimodal systems), code or workflow reliability, and explainability of the intelligence core.

Score bands:

- 0–5: Basic prototype with limited AI implementation.
- 6–10: Functional architecture with moderate technical depth.
- 11–15: Well-structured AI-native system design.
- 16–20: Production-grade engineering maturity suitable for real-world deployment.

3 Business Model (+ Global Readiness)

0–20

20%

Monetization logic, adoption pathway, cross-border applicability. Judges evaluate the clarity of the value proposition, monetization or sustainability pathway, user adoption strategy, cross-border market viability, and evidence of market validation or demand.

Score bands:

- 0–5: Concept without sustainability or business model.
- 6–10: Basic business logic with limited validation.
- 11–15: Viable market pathway with defined users.
- 16–20: Scalable, globally adaptable economic model with strong potential.

4 Real-World Impact (+ Ethical AI Compliance)

0–20

20%

Problem relevance, measurable benefit, responsible AI safeguards. Judges consider the significance of the problem, measurable outcomes or KPIs, societal or economic relevance, responsible data usage, bias mitigation strategies, and overall transparency in AI design.

Score bands:

- 0–5: Weak or unclear impact case.
- 6–10: Clear localized benefit with limited scale.
- 11–15: Strong national relevance with measurable outcomes.
- 16–20: High social return with documented ethical safeguards.

5 Scalability (+ NRB Collaboration)

0–10

10%

Deployment feasibility, modular design, global builder integration. Judges evaluate whether the system can scale technically and operationally, including cloud readiness, modular architecture, infrastructure feasibility, inclusion of NRB or global collaboration, and alignment with international standards.

Score bands:

- 0–3: Demo-only system without scalability considerations.
- 4–7: Early scaling strategy with limited infrastructure planning.
- 8–10: Clear deployment pathway and strong global collaboration readiness.

6 Presentation

0–10

10%

Clarity, structure, confidence, and AI reasoning explanation. Judges assess the clarity of storytelling, effectiveness of the demonstration, ability to explain AI decisions, and overall professionalism and confidence of the team.

Score bands:

- 0–3: Unclear or poorly structured communication.
- 4–7: Clear presentation with moderate structure.
- 8–10: Compelling, data-driven, and investor-ready presentation.

Total Weight

100%